Optimus - Low-Level Design

Version 2.43 — capability and evidence correction, 2026-10-15; reviewed local filing candidate.

Predecessor: Optimus-Cost-Agent-LLD-v2.42.pdf; SHA-256 4727b9ebbc9300f4f8f952468e1e2cf2350aad36ed38592eacfbb43d48661808. v2.42 was filed locally on 2026-10-05 and merged to main through PR #276 (c509fc8, 2026-10-08).

Native carried page bodies originate from Optimus-Cost-Agent-LLD-v2.41.pdf (SHA-256 69400fd474eb30711fcc9a061243d6a4d2e35d39d7794d4aa69f5ff51b98109b), exactly as v2.42 carried them; corrected bodies are replaced in place and the new status section is appended. Every v2.42 page position is accounted for in the correction map.

What this edition changes. For every material capability it separates what is implemented, what has been verified and how, and what is active, at the inspected source 91fd64ac78025efe1076bddb106ff89e6adb2030 (origin/main, 2026-10-15). It changes no accepted requirement, number or historical record. Each correction is listed with its source reference in the correction map filed with the sources (docs/sources/capability-evidence-correction/).

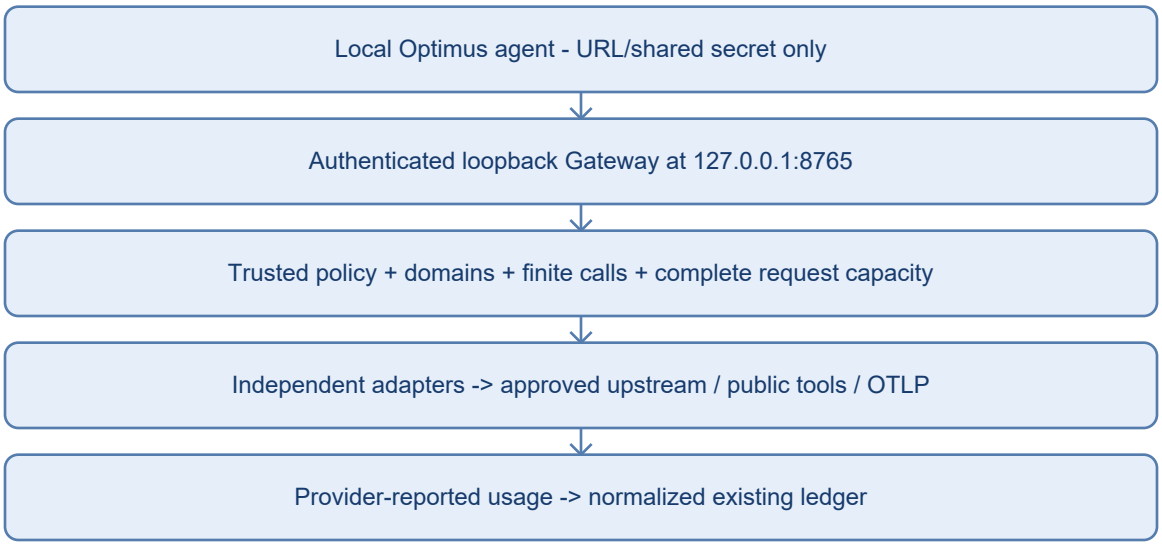
Inspected production facts at 91fd64a. ENFORCEMENT_ACTIVE is False and trusted_snapshot() returns None; no production path constructs ContextAttachment; the shipped alias claude-haiku resolves to anthropic/claude-haiku-4.5; the Gateway web-search provider is Tavily with a Gateway-only key; package/OSV routes still depend on that search configuration; no PyTestArch, maintainability-index, DeepEval, Ragas, PyRIT, Tirith or Vercel dependency, import or test exists; no Rust source exists.

Historical acceptance stays attached to its pins. Offline implementation candidate cf9bb9d (Codex review 2026-10-04); documentation filing 5ee081e; operator acceptance of the offline delivery through 2068366 (2026-10-08); merge c509fc8 (PR #276). The deferred live obligations (Luna route/estimator qualification, genuine summary-quality receipt, real-editor Task 13) remain UNRUN under Plan 12.2 ownership (issue #237). Offline acceptance is not production acceptance, a full CP4 pass or permission to remove the shipped model default.

Authority. Prepared by Claude under the operator's documentation-correction request; Fable 5.1 and Codex review at the plan's checkpoints. This document grants no paid or live calls, activation, push, PR or merge authority, and it was produced without running any product suite.

0. Optimus AI Gateway architecture

The Gateway is a deterministic strict-loopback process. Agent-facing configuration remains only OPTIMUS_GATEWAY_URL and OPTIMUS_API_KEY. The Gateway holds the approved developer aggregator credential in its own local environment or OS credential storage. No hosted Optimus service, OAuth/device flow, tenant wallet, Vault or public origin is introduced. OpenRouter remains the default aggregator. Vercel is an optional second OpenAI-compatible endpoint by design; no Vercel integration exists at 91fd64a. Direct provider adapters remain removed.



Ingress / adapter	Downstream responsibility
/v1/responses; /v1/chat/completions	One approved OpenAI-compatible model transport; preserve distinct input/messages shapes
/v1/tools/web/search	Separate authorized search request. Tavily-backed at 91fd64a (Gateway-only `TAVILY_API_KEY`); the deterministic aggregator-plugin contract in §9C.1 is the target
/v1/tools/web/extract	Bounded HTTPS extract with provenance and revalidation
/v1/tools/package/lookup	Public PyPI/npm/Maven clients; independence from search configuration is required by §9C.4 and not yet implemented
/v1/tools/security/advisory	Public OSV client; same §9C.4 requirement
/v1/observability/traces	Structured trace validation/redaction -> OTel/OTLP -> Phoenix by default

All seven routes authenticate locally. No MCP endpoint is added. Product dollar stops retire only under the accepted migration; independent evaluation ceilings remain.

External component	Preserved edge
OpenRouter	Approved default model transport; separately authorized search is a target
Tavily	Implemented web-search provider at 91fd64a; Gateway-only key; replacement pending
Vercel	Design option; no integration at the inspected source
PyPI / npm / Maven	Independent package APIs
OSV	Independent advisory API
Phoenix / OTLP collector	Sanitized configured trace export
RedisTimeSeries	Existing usage/state attribution

The host owns Agent planning count/time/failure bounds; the Gateway owns enabled trusted request policy and capacity.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

C. Gateway responsibilities

- Authenticate the local agent with a shared secret on strict loopback.
- Resolve model aliases through an approved OpenAI-compatible aggregator.
- Broker web search (Tavily at 91fd64a; deterministic aggregator plugin is the target), bounded extract, package lookup, and OSV advisory; route independence from search configuration is a §9C.4 requirement not yet met.
- Enforce execution mode, tool policy, domain rules, call caps, provenance, complete request capacity and finite work.
- Persist run/session/request/Gateway/provider request attribution.
- Isolate upstream credentials from the agent process.
- Accept structured trace ingress, map it to OTel, and export OTLP.

D. Gateway-facing API shape

```
# Responses API shape - top-level "input"
POST /v1/responses

# Chat Completions shape - "messages" array
POST /v1/chat/completions

# Typed tools
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory

# Authenticated structured trace ingress
POST /v1/observability/traces
```

All routes authenticate with the local shared secret. Trace ingress is operational, not a model/tool billing route. No MCP endpoint is added.

E. Process-scoped configuration boundary

Agent process	Gateway process
`OPTIMUS_GATEWAY_URL`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER=openrouter`
`OPTIMUS_API_KEY`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY`
	`TAVILY_API_KEY` (Gateway-only; current search provider)
No upstream or OTel key	`OPTIMUS_LOCAL_GATEWAY_SHARED_SECRET`
Strict loopback URL	domains, Redis URL, optional OTLP endpoint
No override surface	no hosted origin, tenant profile, or production mode

0A. Local vs. Gateway Configuration and Provider-Cost Mapping

Phase 1 runtime rule

The agent runtime allows only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`. The Gateway owns the developer's approved aggregator credential. Local agent-side Tavily, OpenAI, OpenRouter, GLM, LangSmith, or other provider keys are rejected.

Provider-cost mapping

`OPTIMUS_API_KEY` is a local shared secret, not a wallet key. The developer funds the aggregator account directly. The Gateway records the aggregator response into one local ledger.

Call class	Required accounting
Model completion	aggregator, actual provider/model when returned, tokens/cache, billing units, provider-reported `cost_usd`, request IDs
Web search	Tavily provider at 91fd64a: provider-native `tavily_credits` units recorded as `ProviderUsage` with request identity; the deterministic aggregator plugin (separate minimal model call, structured citations, search-use accounting) is the target
Package/advisory	operational request evidence; no fabricated provider cost
Trace export	delivery state and trace IDs; no allocated/amortized request charge

```
Agent shared secret
  -> loopback Gateway trust, capacity and accounting
  -> developer aggregator account
  -> provider-reported usage and USD cost
  -> GatewayUsage + normalized local ledger
```

The USD rename is complete at 91fd64a: ACP result payloads carry `ledger_run_total_cost_usd` and `ledger_run_total_billing_units` (`optimus/acp/dispatcher.py`), and no credit-named ledger field remains under `src/`. The rename changed no USD semantics and introduced no cross-run policy.

Gateway Tool and Observability Endpoints

The agent never calls OpenRouter, Vercel, package registries, OSV, Phoenix, or another OTLP backend directly. It calls the loopback Gateway's typed endpoints.

```
POST /v1/responses
POST /v1/chat/completions
POST /v1/tools/web/search
POST /v1/tools/web/extract
POST /v1/tools/package/lookup
POST /v1/tools/security/advisory
POST /v1/observability/traces
```

Search is an authorized, separate, minimal OpenRouter plugin request. Package and advisory routes remain independent of the paid-search configuration.

The canonical observability path is:

```
agent structured trace event
  -> authenticated /v1/observability/traces
  -> Gateway validation and redaction
  -> OpenTelemetry span/event mapping
  -> OTLP export
  -> Phoenix by default
```

The agent has no backend credential or direct backend egress. LangSmith and amortized observability-cost accounting are not part of this architecture. The route list has no MCP endpoint.

1. ACP Protocol Framing & JSON-RPC Contract Specification

To prevent streaming fragmentation errors, the ACP interface enforces a strict HTTP-style byte framing header layout. Malformed payloads trigger immediate JSON-RPC standard error blocks.

A. Header, Buffer Size Limits & Serialization Standards

Every frame must prepend an explicit Content-Length prefix sequence. To prevent Denial of Service (DoS) attacks, the transport plane enforces a strict Maximum Header Size of 8 KB and a Maximum Body Size of 10 MB. Responses echo a matching structural correlation integer via the id field.

```
[INBOUND SYSTEM STREAM TRANSACTION]
Content-Length: 114

{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "agent/prompt",
  "params": {
    "prompt": "Refactor...",
    "targetFile": "main.py"
  }
}
```

```
PARSE_ERROR = {
    "code": -32700,
    "message": "Parse error: Invalid JSON payload received."
}
INVALID_REQUEST = {
    "code": -32600,
    "message": "Invalid Request: Schema mapping parameters missing."
}
INTERNAL_ERROR = {
    "code": -32603,
    "message": "Internal error: Underlying task execution failure."
}
ID_COLLISION_ERROR = {
    "code": -32001,
    "message": "Server error: Request ID collision detected."
}
```

2. Cross-Platform Stream Transport Layer

```
import asyncio
import sys
import json
import re
from typing import Optional, Tuple
from json import JSONDecodeError
from concurrent.futures import ThreadPoolExecutor

class CrossPlatformStreamReader:
    def __init__(self, max_header_bytes: int = 8192,
                 max_body_bytes: int = 10485760):
        self.executor = ThreadPoolExecutor(max_workers=1)
        self.max_header = max_header_bytes
        self.max_body = max_body_bytes

    def _sync_blocking_read(self) -> Tuple[Optional[bytes], bool]:
        header = b""
        while b"\r\n\r\n" not in header:
            if len(header) >= self.max_header:
                return None, True
            char = sys.stdin.buffer.read(1)
            if not char:
                return b"", False
            header += char
        match = re.search(b"(?i)Content-Length\s*:\s*(\d+)", header)
        if not match or int(match.group(1)) > self.max_body:
            return None, True
        content_length = int(match.group(1))
        return sys.stdin.buffer.read(content_length), False

    async def read_frame(self) -> Tuple[Optional[dict], Optional[dict]]:
        loop = asyncio.get_running_loop()
        raw_bytes, bad_frame = await loop.run_in_executor(
            self.executor, self._sync_blocking_read
        )
        if bad_frame:
            return None, PARSE_ERROR
        if not raw_bytes:
            return None, None
        try:
            return json.loads(raw_bytes.decode("utf-8")), None
        except JSONDecodeError:
            return None, PARSE_ERROR
```

3. Queued Task Lifecycle, Backpressure Controls & Runtime Pooling

```
import httpx
import uuid
import redis.asyncio as aioredis

class ManagedTaskManager:
    def __init__(self, max_concurrent_prompts: int = 3,
                 execution_timeout_secs: int = 120):
        self.semaphore = asyncio.Semaphore(max_concurrent_prompts)
        self.timeout = execution_timeout_secs
        self.all_tracked_tasks = {}
        self.http_pool = None
        self.redis_pool = None

    async def initialize_runtime_pools(self, redis_url: str):
        if self.all_tracked_tasks:
            raise RuntimeError(
                "Cannot reinitialize pools while tasks are running."
            )
        if self.http_pool or self.redis_pool:
            if self.http_pool:
                await self.http_pool.aclose()
            if self.redis_pool:
                await self.redis_pool.disconnect()
        self.http_pool = httpx.AsyncClient(
            limits=httpx.Limits(
                max_connections=10, max_keepalive_connections=5
            ),
            timeout=httpx.Timeout(30.0),
        )
        self.redis_pool = aioredis.ConnectionPool.from_url(
            redis_url, max_connections=20, decode_responses=True
        )
```

```
    async def enqueue_and_execute(self, request_id: Optional[int],
                                  pipeline_coro_factory) -> dict:
        active_id = request_id if request_id is not None \
            else int(uuid.uuid4().int & 0x7FFFFFFF)
        if not self.http_pool or not self.redis_pool:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32003,
                    "message": "Server error: Connection pools uninitialized.",
                },
            }
        if active_id in self.all_tracked_tasks:
            return {
                "status": "ERROR",
                "error": {
                    "code": -32001,
                    "message": "Server error: Request ID collision detected.",
                },
            }

        async def _orchestrated_run():
            async with self.semaphore:
                coro = pipeline_coro_factory(self.http_pool, self.redis_pool)
                return await asyncio.wait_for(coro, timeout=self.timeout)

        loop = asyncio.get_running_loop()
        wrapper_task = loop.create_task(_orchestrated_run())
        self.all_tracked_tasks[active_id] = wrapper_task
        try:
            result = await wrapper_task
            return {"status": "SUCCESS", "data": result}
        except asyncio.CancelledError:
            return {
                "status": "ERROR",
                "error": {"code": -32603, "message": "Task cancelled during shutdown."},
            }
        except asyncio.TimeoutError:
```

```
                "error": {"code": -32603, "message": "Execution timeout."},
            }
        finally:
            self.all_tracked_tasks.pop(active_id, None)

    async def graceful_shutdown(self):
        for task in list(self.all_tracked_tasks.values()):
            task.cancel()
        if self.all_tracked_tasks:
            await asyncio.gather(
                *self.all_tracked_tasks.values(), return_exceptions=True
            )
        self.all_tracked_tasks.clear()
        if self.http_pool:
            await self.http_pool.aclose()
        if self.redis_pool:
            await self.redis_pool.disconnect()
```

4. Behavioral Governance: modes, strategies and scope

The following representative contracts preserve the existing scope and mode vocabulary while retiring the HAIKU/PRO model-tier enum. Executable implementations remain in src/; these type shapes are design contracts.

Contract	Values / fields
ExecutionMode	PLAN; CHAT; AGENT. This existing vocabulary retains the read-only Plan/Chat boundary.
CodeGenerationScope	INLINE_SNIPPET; PATCH_PROPOSAL; FILE_MUTATION; MULTI_FILE_CHANGESET
ExecutionStrategy	DIRECT; PLAN_AND_EXECUTE; REACT; REFLECTION
RigorLevel	LOW; MEDIUM; HIGH
Model tier	ultra-cheap; cheap; review. Review role is dormant; use the trusted registry, not HAIKU/PRO aliases.
ToolOperationKind	READ; WRITE; EXTERNAL_MUTATION
OptimusToolErrorCode	VALIDATION_ERROR; RESOURCE_ERROR; PROVIDER_ERROR; TIMEOUT_ERROR
OptimusToolError	code; message; optional context. Exception keeps those fields and its message.
AssumptionItem	claim; basis; confidence default medium; requires_verification default true
AssumptionLedger	assumptions list, empty by default; no new conversation claim-tracking subsystem
ToolResponse	is_success; payload empty dict by default; optional error_code/error_msg

ToolResponse.to_jsonrpc_error maps VALIDATION_ERROR to -32002, RESOURCE_ERROR to -32004, PROVIDER_ERROR to -32006, TIMEOUT_ERROR to -32005, and unmapped/no typed code to -32603. Return code, Tool Execution Failure message, and data containing typed error_code and context. These fields are retained across the successor.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

4. Scope classification and strategy selection (continued)

The following retained prototype illustrates scope classification; it is not a new runtime activation or additional policy decision. Ordered branches:

Priority	Condition	Scope
1	Shared module touched; multiple directory roots; structural create/delete across more than one file; or more than three files	MULTI_FILE_CHANGESET
2	Structural create/delete; or one file with more than 100 lines	FILE_MUTATION
3	More than 15 lines	PATCH_PROPOSAL
4	Otherwise	INLINE_SNIPPET

Execution strategy remains harness-controlled. Start with at most five tool calls and zero reflection passes; a validation failure or shared-module change selects bounded Reflection with at most 15 calls and three passes. High task risk or a multi-file changeset selects Plan-and-Execute with at most ten calls and one reflection. High ambiguity otherwise selects ReAct with at most 12 calls. Other work uses Direct execution. No new mandatory reflection loop is introduced. After approved trusted policy activation, all branches obtain the eligible fixed model from the trusted role/route registry and obey its actual request capacity, with a total context ceiling of 262144 including one output reserve. Retire the old branch-specific 100000/150000/250000 token ceilings and HAIKU/PRO aliases. History strategy allocations and final input/output packing are separate. Auto classification and struggle escalation are not activated. Mutation enforcement continues in section 4: PLAN cannot perform WRITE or EXTERNAL_MUTATION and raises the typed validation error. The existing MutationGuard wraps that assertion before execution; the continuation on the next page retains its path-containment contracts.

4. Mutation assertion and path containment (retained prototype)

The full assertion is shown here so the raise statement is not orphaned after the preceding page replacement. ExecutionMode is the retained PLAN/CHAT/AGENT vocabulary. These illustrative contracts preserve the original containment checks; executable implementation remains authoritative.

```
def assert_mutation_allowed(mode: ExecutionMode,
                            operation: ToolOperationKind, tool_name: str):
    if mode != ExecutionMode.AGENT and operation in {
        ToolOperationKind.WRITE, ToolOperationKind.EXTERNAL_MUTATION
    }:
        raise OptimusToolError(
            OptimusToolErrorCode.VALIDATION_ERROR,
            f"Access security violation error: "
            f"Mutation blocked in Plan/Chat mode: {operation}",
        )
class MutationGuard:
    def __init__(self, mode: ExecutionMode):
        self.mode = mode
    def enforce_gate(self, op_kind: ToolOperationKind, tool_name: str):
        assert_mutation_allowed(self.mode, op_kind, tool_name)
class ASTOptimizationInput(BaseModel):
    project_path: DirectoryPath = Field(
        ..., description="Absolute path to the project root."
    )
    target_file: str = Field(..., description="Relative file path string.")
    adl_rules: list[str] = Field(default_factory=list)
    execution_mode: ExecutionMode = Field(default=ExecutionMode.PLAN)
    generation_scope: CodeGenerationScope = Field(
        default=CodeGenerationScope.INLINE_SNIPPET
    )
    rigor_budget: RigorLevel = Field(default=RigorLevel.LOW)
    assumption_ledger: AssumptionLedger = Field(
        default_factory=AssumptionLedger
    )
    @model_validator(mode="after")
    def validate_paths_and_safeguards(self) -> "ASTOptimizationInput":
        if Path(self.target_file).is_absolute():
            raise ValueError("Target file must be a relative path configuration.")
        try:
            resolved_base = Path(os.path.abspath(self.project_path)).resolve(
                strict=True
            )
            candidate_path = Path(os.path.abspath(
                os.path.join(str(resolved_base), self.target_file)
            ))
        except (ValueError, FileNotFoundError, RuntimeError):
            raise ValueError("Malformed or invalid project root directory specification.")
        try:
            if candidate_path.exists() or candidate_path.is_symlink():
                resolved_target = candidate_path.resolve(strict=False)
                if os.path.commonpath([
                    str(resolved_base), str(resolved_target)
                ]) != str(resolved_base):
                    raise ValueError(
                        "Breakout attempt detected via target file path alignment."
                    )
            else:
                parent_path = candidate_path.parent
                resolved_parent = parent_path.resolve(strict=True)
                if os.path.commonpath([
                    str(resolved_base), str(resolved_parent)
                ]) != str(resolved_base):
                    raise ValueError(
                        "Symlink ancestry breakout via target parent folder."
                    )
        except ValueError as ve:
            raise ValueError(
                f"Cross-drive partition boundary constraint or layout failure: {ve}"
            )
        except (FileNotFoundError, RuntimeError):
            raise ValueError(
                "Missing or invalid ancestral directory infrastructure mapping."
            )
        return self
```

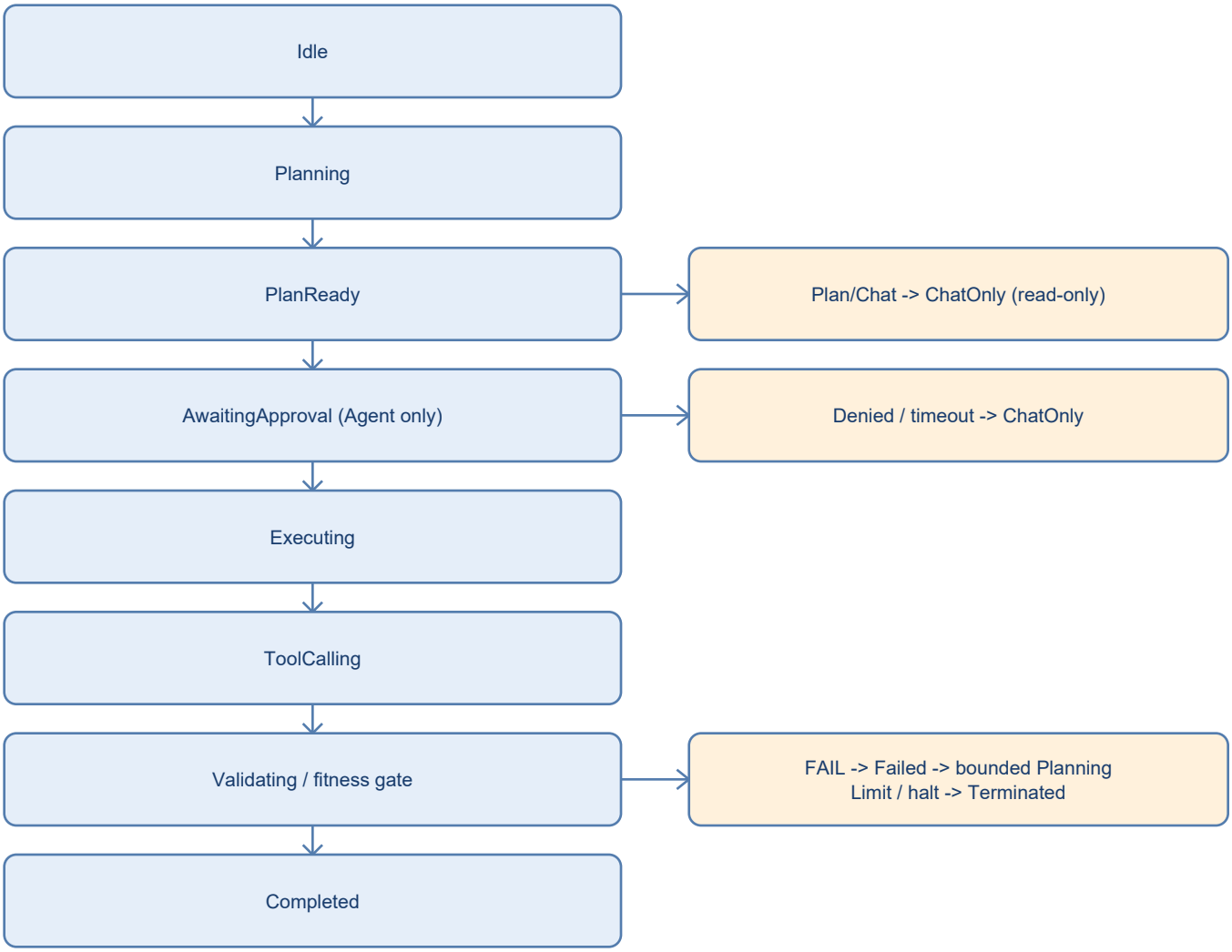
4A. Agent State Model and Execution Pathways

This section defines the complete agent lifecycle — from initial idle state through planning, execution, tool calling, and termination — and specifies which transitions are valid, which are rejected deterministically, and what permissions each mode grants.

Lifecycle State Machine

The agent operates as a deterministic state machine. Every state transition is validated at the control-plane boundary; invalid transitions are rejected with a structured -32002 error before any side-effectful work begins.

4A. Agent lifecycle state machine



Branch	Required transition
Plan/Chat	PlanReady -> ChatOnly; no mutation
Agent	PlanReady -> AwaitingApproval; current exact approval -> Executing
Tool result	ToolCalling -> Validating; PASS -> Executing or Completed
Fitness failure	Failed -> Planning only within finite count/time/failure bounds
Exhaustion / cancellation	Failed -> Terminated with true reason and retained effects/receipts

No direct PlanReady -> Executing transition is allowed. Historical approval and model prose cannot bypass the current artifact gate. This figure does not activate Auto routing.

4A. Valid and invalid transitions

Every transition is deterministic; model text cannot override a rejection. Existing mode and exact-approval checks remain required.

From	To	Permitted / condition
Idle	Planning	Yes, user request
Planning	PlanReady	Yes, requirements and mode resolved
PlanReady	ChatOnly	Yes, Plan/Chat; no mutation
PlanReady	Executing directly	No; AwaitingApproval cannot be bypassed
PlanReady	AwaitingApproval	Yes, Agent needs exact artifact approval
AwaitingApproval	Executing	Yes, current approval and target checks pass
AwaitingApproval	ChatOnly	Yes, denied/timed-out approval returns advisory outcome
Executing	ToolCalling	Yes, ToolInvocationPolicy authorizes the call
Executing	Mutating ToolCalling	Only Agent mode plus approval/guard controls
ToolCalling	Validating	Yes, operation response received
Validating	Executing	Yes, applicable gates pass
Validating	Failed	Yes, record actual failure and count it
Failed	Planning	Only within configured count/time/failure controls; inject targeted failure context
Failed	Terminated	Finite work exhausted, cancellation or permanent failure; report cause. No product dollar exhaustion after migration.
Executing	Completed	All planned work/gates complete; actual effect evidence retained
Any	Executing by bypass	No, -32002; approval path remains mandatory

Interim ACP defaults remain three planning rounds, 30 minutes and two repeated failures. Generic architecture diagrams do not override concrete configured execution limits. Independent evaluation budget exhaustion is a separate scoped stop, not product policy.

Mode-specific permission matrix

Permissions are enforced by the harness, not model instruction. Agent permissions still require the current artifact, approval and tool-specific guard; a Yes cell is not unconditional authority.

Capability	Plan/Chat	Agent	Enforcement
Read repo / inspect files	Yes	Yes	Existing eligible workspace / read controls
Read-only analysis tools	Yes	Yes	Existing read controls
Policy-gated web search / extract	Yes	Yes	ToolInvocationPolicy and provenance
Produce diffs / snippets as text	Yes	Yes	Text payload only, not applied
Shadow apply patch	No	Yes	assert_mutation_allowed gate
Mutate working tree	No	Yes	assert_mutation_allowed + MutationGuard + approval
Shell / build / test	No	Yes	Agent mode and applicable fitness/approval gates
Append cost / audit telemetry	Yes	Yes	Append-only
Modify telemetry / ledger entries	No	No	Immutable after write

Execution pathway

The retained pathway connects captured user intent/mode, explicit eligible model selection, mutation gating and telemetry through the Gateway boundary. Intent classification in the predecessor was a reserved capability; this successor does not activate Auto/classifier routing. Section 4A's revised pathway and state diagram preserve advisory ChatOnly, AwaitingApproval, Failed/replan, fitness and termination branches.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

4A. Execution pathway

Stage	Deterministic boundary
IDE / ACP ingress	User prompt, mode/config capture and session identity
Host selection	Explicit eligible role/model and trusted effective registry; Auto/Haiku triage removed
Mode and approval	Chat stays read-only; Agent needs exact artifact approval before effectful tools
Feedforward assembly	Bounded workspace evidence and Redis state plus the optional Context Engine; exact authority. AST slicing and ADL constraint controls are intended, not established at 91fd64a (§14)
Final Gateway guard	Strict loopback, shared secret, approved endpoint, full input and one enforced output reserve per actual attempt
Upstream generation	Gateway-owned aggregator credential; no Vault, hosted tenant or provider key in CORE
Feedback	Complete finish required for executable candidate; shadow validation and fitness gates
Mutation gate	Current approved artifact/target, permission and start gates; no model authority
Telemetry / settlement	Actual provider receipts, exact effects, exactly-once costs and explicit unknowns; no fabricated charge

This is the complete updated pathway. Existing lifecycle and mode-matrix sections remain. Attached test composition does not establish normal bootstrap activation; section 13 specifies the release prerequisites.

4A. Execution strategy selection and tool policy

Strategy	Trigger	Behavior / bounds
Direct	Low rigor, no mutation	Single result, up to five tool calls, zero reflection; eligible fixed model
Plan-and-Execute	High task risk or multi-file changeset (retained prototype)	Structured plan and exact approval; up to ten calls and one reflection
ReAct	High ambiguity	Interleave bounded reasoning/observations; preserve configured call/iteration bounds
Reflection	Gate failure or shared/core change	Targeted recovery, at most three reflection passes; never the default loop

These descriptive strategies do not automatically activate Auto routing or override current three-round/30-minute/two-repeated-failure ACP limits. All obey final verified route capacity; HAIKU/PRO aliases and separate risk-tier context ceilings are retired. Tool invocation is harness-controlled, first matching policy wins. Reject mutating tools in Plan/Chat with -32002 before I/O. WEB_SEARCH requires a current/external-fact, API or security signal and a valid reason code; local evidence remains first. WEB_EXTRACT requires the URL to belong to the prior authorized search result set. A mutation without current artifact approval is rejected before execution. Local read/inspect/analysis tools still pass the existing deterministic mode/permission rules. The model never selects the policy row or invents provenance.

4A. Failure and retry lifecycle

Record the true failure, preserve receipts/effects and inject bounded failure context only when configured work limits permit another planning round. Keep count/time/repeated-failure controls distinct from provider attempts. The interim ACP defaults are three planning rounds, 30 minutes and two repeated failures. Product BudgetExhaustedError is retired by the accepted migration; independent evaluations may retain their scoped dollar stop. The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

5. ADL parser continuity and architectural evolution

ADLParserStrategy retains parse_and_validate(adl_rules: list[str], proposed_diff: str) -> tuple[bool, list[dict]] as its abstract validation contract. PrototypeRegexADLParser retains the existing ASSERT(<a>) HAS NO DEPENDENCY ON <b> matching grammar, lowercases captured names for code import checks, and checks both an import <b> occurrence and the org.springframework.<b> substring in the diff. Violations retain rule, Structural Pollution metric and Violation: (<a> -> <b>) value. It returns true only when the violations list is empty. The regex implementation is a prototype and does not become a new semantic analyzer through this migration.

6. Resilient Aggregator Calling Layer and Tenacity Rules

All model completions route from the agent to the strict-loopback Gateway using the local shared secret. The Gateway resolves an Optimus model alias to an aggregator model identifier and calls the approved OpenAI-compatible upstream: OpenRouter by default (Vercel AI Gateway is a design option with no integration at 91fd64a).

The aggregator credential exists only in Gateway-owned local configuration or OS credential storage. There is no Vault, hosted Optimus account, or direct single-provider adapter.

Two agent-facing shapes, one upstream transport

- `/v1/responses` accepts the Responses API `input` field.
- `/v1/chat/completions` accepts the Chat Completions `messages` array.
- The validator rejects `messages` at `/v1/responses` and `input` at `/v1/chat/completions`.
- Both paths converge on one authenticated completion service.
- The surviving transport is `UrllibOpenAICompatibleClient` in `src/optimus_gateway/upstream_client.py` (built by `optimus_gateway.providers.build_upstream_client`).

The legacy host permits an initial attempt plus at most three retries (`optimus/retry/policy.py`, `max_retries = 3`; the enforced-policy limit is `MAX_ROUTE_ATTEMPTS = 2` in `optimus_model_policy/binding.py`). The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

Provider-reported `usage.cost` is authoritative in the corrected architecture. Missing, null, negative, or malformed usage/cost fails closed before generated output is accepted.

6.1 OpenAI-compatible upstream pattern

```
class UrllibOpenAICompatibleClient:
    """Gateway-owned aggregator transport.

    The agent never receives the upstream URL or credential. The transport
    has no direct-provider selection branch.
    """

    def __init__(self, *, api_key: str, base_url: str) -> None:
        self._api_key = api_key
        self._base_url = base_url.rstrip("/")

    def create_message(
        self, *, model: str, input_text: str
    ) -> ProviderMessageResult:
        payload = {
            "model": model,
            "messages": [{"role": "user", "content": input_text}],
        }
        body = self._post_json("/chat/completions", payload)
        return parse_openai_compatible_message_and_usage(body)
```

`parse_openai_compatible_message_and_usage` must validate output, provider request identity, billing units, token/cache detail, resolved model/version when present, and provider-reported USD cost. The Gateway returns the requested agent-facing shape plus the same validated GatewayUsage contract.

```
Agent /v1/responses "input" --------\
                                     > completion service
Agent /v1/chat/completions "messages"/       |
                                             v
                        OpenAI-compatible aggregator transport
```

The upstream transport may use Chat Completions while the Gateway preserves both agent-facing contracts. Those shapes must never be mixed.

7. Composite Fitness Engine Scaffolding

```
class FitnessGateInterface(ABC):
    @abstractmethod
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        pass

class DependencyRuleGate(FitnessGateInterface):
    def __init__(self, parser: ADLParserStrategy):
        self.parser = parser

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        passed, violations = self.parser.parse_and_validate(
            state.get("adl_rules", []), state.get("generated_patch", "")
        )
        return passed, {"violations": violations}

class PlaceholderMetricGate(FitnessGateInterface):
    def __init__(self, max_complexity: int = 15):
        self.max_complexity = max_complexity

    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "metrics": {
                "cyclomatic_complexity":  4,
                "maintainability_index": 82,
            }
        }

class PlaceholderTestArchGate(FitnessGateInterface):
    def evaluate_gate(self, state: dict) -> Tuple[bool, dict]:
        return True, {
            "testarch_status": "COMPLIANT",
            "graph_cyclical":  False,
        }

class CompositeFitnessEngine:
    def __init__(self, gates: list[FitnessGateInterface]):
        self.gates = gates

    def execute_harness_check(self, state: dict) -> dict:
        combined_telemetry = {}
        for gate in self.gates:
            passed, telemetry_delta = gate.evaluate_gate(state)
            combined_telemetry.update(telemetry_delta)
            if not passed:
                return {
                    "fitness_verdict": "FAIL",
                    "telemetry":       combined_telemetry,
                }
        return {
            "fitness_verdict": "PASS",
            "telemetry":       combined_telemetry,
        }
```

8. Patch Workspace Lifecycle Boundaries

Applying patches exposes systems to directory traversal attacks. Optimus normalises boundaries using strict commonpath comparisons and strictly isolates validation steps entirely inside a localised shadow workspace environment.

- **Shadow Workspace Environment**: Patches are written exclusively to a workspace-local shadow directory located at .optimus/shadow_env/. The patch diff is programmatically parsed, applied, and validated against the duplicate code tree via python file mutation loops.
- **Operating Mode Enforcement Contra**ct: If execution_mode equals AgentExecutionMode.PLAN, tool operations matching ToolOperationKind.WRITE or ToolOperationKind.EXTERNAL_MUTATION are completely bypassed and blocked from modifying local structures. Read-only inspection tools remain fully authorised.
- **Atomic Apply to Working Tree & Surfacing Rejec**tions: Modifications are applied to the working tree only after a clean PASS verdict, guaranteeing transaction atomicity and immediate rollback safety on any system fault. If a gate trips a FAIL signature, the patch is rejected, the transaction aborts, and the duplicate shadow environment is cleanly purged.

9. Tool Registry, Invocation Policy & Evidence Ledger

Tool use is policy-driven first, model-requested second: the model may request evidence, but the harness alone decides whether a tool is permitted, necessary, and cost-justified for the active execution mode and rigor tier. This section defines the typed tool registry, the deterministic invocation policy matrix, and the Evidence Ledger schema that records every external lookup alongside the Assumption Ledger.

A. Tool Class Enum & Registry Contract

```
class ToolClass(str, Enum):
    LOCAL_REPO_DISCOVERY         = "LOCAL_REPO_DISCOVERY"
    VALIDATION_GATE              = "VALIDATION_GATE"
    PATCH_WORKSPACE              = "PATCH_WORKSPACE"
    COST_TELEMETRY               = "COST_TELEMETRY"
    WEB_EVIDENCE                 = "WEB_EVIDENCE"
    PACKAGE_AND_ADVISORY_METADATA = "PACKAGE_AND_ADVISORY_METADATA"
    EXTERNAL_EXECUTION           = "EXTERNAL_EXECUTION"

class EvidenceReasonCode(str, Enum):
    API_DOCS_OUTDATED = "API_DOCS_OUTDATED"
    CURRENT_FACT      = "CURRENT_FACT"
    SECURITY_ADVISORY = "SECURITY_ADVISORY"
    USER_REQUESTED    = "USER_REQUESTED"
    PACKAGE_VERSION   = "PACKAGE_VERSION"

class SearchDepth(str, Enum):
    """Internal rigor presets. NOT assumed to equal the gateway's wire
    values 1:1 -- see TAVILY_SEARCH_DEPTH_MAP in section C, which is
    the only place an internal value is translated into the string
    actually sent to the provider via the gateway."""
    BASIC    = "basic"
    FAST     = "fast"
    ADVANCED = "advanced"

class ToolRegistryEntry(BaseModel):
    tool_name:             str
    tool_class:            ToolClass
    allowed_modes:         list[AgentExecutionMode]
    requires_policy_trigger: bool = False
    is_mutating:           bool = False
    max_calls_per_run:     Optional[int] = None

class ToolRegistry(BaseModel):
    entries:     dict[str, ToolRegistryEntry] = Field(default_factory=dict)
    # In-process call counters, keyed by f"{run_id}:{tool_name}". Single
    # process / single-instance only -- see the Redis-backed variant
    # below for concurrent or multi-process execution.
    call_counts: dict[str, int] = Field(default_factory=dict)

    def check_static_policy(
        self, entry: Optional[ToolRegistryEntry],
        mode: AgentExecutionMode, policy_triggered: bool,
    ) -> bool:
        if entry is None:
            return False
        if mode not in entry.allowed_modes:
            return False
        if entry.requires_policy_trigger and not policy_triggered:
            return False
        return True
```

```
    def is_allowed(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """READ-ONLY inspection (e.g. UI previews, dry-run explainers).
        Does not increment the call counter. Never use this to gate an
        actual tool invocation -- use authorize_and_record_call() below,
```

```
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        if entry.max_calls_per_run is not None:
            counter_key  = f"{run_id}:{tool_name}"
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        return True

    def authorize_and_record_call(
        self, tool_name: str, mode: AgentExecutionMode,
        policy_triggered: bool, run_id: str,
    ) -> bool:
        """The only method that should gate an actual tool call. Checks
        the static policy and the call-count ceiling, then increments
        the counter, as a single atomic step -- so authorization and
        bookkeeping can never drift apart, and a caller cannot forget
        to record a call it was just allowed to make."""
        entry = self.entries.get(tool_name)
        if not self.check_static_policy(entry, mode, policy_triggered):
            return False
        counter_key = f"{run_id}:{tool_name}"
        if entry.max_calls_per_run is not None:
            calls_so_far = self.call_counts.get(counter_key, 0)
            if calls_so_far >= entry.max_calls_per_run:
                return False
        self.call_counts[counter_key] = (
            self.call_counts.get(counter_key, 0) + 1
        )
        return True
```

The in-process dict above is sufficient for Phase 1's single-instance daemon, but is not safe across multiple worker processes and does not survive a restart mid-run. The production-grade path swaps the in-memory counter for a Redis-backed atomic increment, keyed identically and protected by a TTL matching the run's expected lifetime:

```
async def authorize_and_record_call_redis(
    registry: ToolRegistry, redis_client: aioredis.Redis,
    tool_name: str, mode: AgentExecutionMode,
    policy_triggered: bool, run_id: str, run_ttl_secs: int = 3600,
) -> bool:
    entry = registry.entries.get(tool_name)
    if not registry.check_static_policy(entry, mode, policy_triggered):
        return False
    if entry.max_calls_per_run is None:
        return True
    counter_key = f"toolcalls:{run_id}:{tool_name}"
    # INCR is atomic at the Redis level, so concurrent callers cannot
    # race past the ceiling the way two in-process checks could.
    new_count = await redis_client.incr(counter_key)
    if new_count == 1:
        await redis_client.expire(counter_key, run_ttl_secs)
    if new_count > entry.max_calls_per_run:
        await redis_client.decr(counter_key)
        return False
    return True
```

B. Deterministic Invocation Policy Matrix

The policy matrix maps task signals to permitted tool classes. The model never selects a tool class directly; it raises a typed signal and the harness resolves the matching policy row. PACKAGE_AND_ADVISORY_METADATA deliberately covers both package-registry lookups (PyPI, npm, Maven Central) and security-advisory lookups (OSV.dev, GitHub advisories): both are read-only, policy-triggered, and keyed by a package or CVE identifier, so a single tool class keeps the registry simple while the signal name (TASK_TOUCHES_DEPENDENCIES vs. TASK_TOUCHES_SECURITY_CVE) still makes the routing intent explicit.

9C. Typed Evidence Acquisition Wrappers

The agent authenticates only to the loopback Gateway. Search-provider mechanics remain behind `/v1/tools/web/search`; extract, package, and advisory capabilities have independent dependencies.

Strict-loopback settings

```
class OptimusGatewaySettings(BaseModel):
    gateway_url: str = "http://127.0.0.1:8765"
    optimus_api_key: SecretStr

    @model_validator(mode="after")
    def validate_loopback_gateway(self) -> "OptimusGatewaySettings":
        parsed = urlsplit(self.gateway_url)
        if parsed.scheme not in {"http", "https"}:
            raise ValueError("Gateway URL must use HTTP(S)")
        if parsed.username is not None or parsed.password is not None:
            raise ValueError("Gateway URL must not contain userinfo")
        if parsed.hostname not in {"127.0.0.1", "localhost", "::1"}:
            raise ValueError("Gateway URL must resolve to strict loopback")
        return self
```

There is no production mode, built-in hosted origin, extra-origin override, signed tenant profile, or non-loopback trust seam. The final source must remain aligned with the separately reviewed strict-loopback implementation.

9C.1 Deterministic search contract (target) and the implemented provider

Implemented at 91fd64a: the Gateway's `/v1/tools/web/search` route is served by `TavilyWebToolProvider` (`optimus_gateway/tool_providers.py`), configured with the Gateway-only `TAVILY_API_KEY`; its usage is recorded as provider-native `tavily_credits`. The contract below is the accepted target for replacing that provider with a dedicated aggregator request; it is not implemented in the Gateway.

The target converts an authorized search request into a dedicated OpenRouter Chat Completions request using one cheap Flash/Haiku-class model, low `max_tokens`, and `plugins: [{"id": "web"}]`. Search is never attached to the primary generation call.

```
{
  "model": "google/gemini-2.5-flash-lite",
  "messages": [{"role": "user", "content": "<minimal search request>"}],
  "max_tokens": 16,
  "plugins": [{
    "id": "web",
    "max_results": 3,
    "search_prompt": "<policy-bounded query>"
  }]
}
```

The Gateway would forward the effective domain policy, parse only `annotations[].url_citation`, independently revalidate every returned URL, and copy upstream usage/cost. Assistant prose and annotation character offsets are not evidence.

Measured baseline (architecture spike, 2026-07-27)

Measurement	Value	Scope
Minimal-call annotations present	3/3	`tools/run_openrouter_web_search_spike.py`, run against OpenRouter directly
Include-domain violations	0	same spike
Exclude-domain violations	0	same spike
Citations structurally complete	9/9	same spike
Mean latency per search	7,068.7 ms	same spike; one host, one day
Mean reported cost	$0.0051584	same spike; prices of that day

These values were produced by the spike tool, not by a Gateway test, and the spike is an experimental measurement harness, not production Gateway code. Three to five sequential searches would project to 21.2-35.3 seconds at that latency. The supplied Tavily comparison (about 1-2 seconds and $0.008/search) was not measured in this spike. The deterministic aggregator plugin is deprecated by OpenRouter (§9C.2), so the target itself is conditional.

9C.2 Plugin deprecation and successor gate

The deterministic OpenRouter plugin is deprecated. A live release probe must block release on removal or behavior change.

The first successor evaluation is a dedicated server-tool request with:

- `max_uses: 1`;
- a one-call budget;
- mandatory search-use accounting;
- valid structured annotations;
- domain-policy enforcement;
- fail-closed behavior when execution evidence is absent.

This is verified-or-fail, not deterministic by construction. If it fails, the fallback is a standalone search provider with a second credential and balance.

9C.3 Bounded direct extract

Extract performs a Gateway-side HTTPS fetch only for URLs recorded by a prior approved search in the same run. It:

- revalidates every redirect and final host;
- rejects URI userinfo and private, link-local, or loopback resolution;
- bounds connect/read time, bytes, redirects, and extracted characters;
- accepts only supported textual media;
- treats HTML and extracted text as untrusted input;
- never executes page content.

The spike fetched 270,008 bytes and parsed 60,077 characters in 589.5 ms. A 1,000,000-byte limit failed closed on a legitimate larger page; production limits and streaming behavior require explicit tests.

9C.4 Independent tool dependency construction

Status at 91fd64a: `optimus_gateway.providers.build_tool_dependencies` builds one `GatewayToolDependencies` bundle (web, package, advisory providers, tool policy, Redis tool state and the per-tool call cap) and returns `None` when the Tavily key, the tool Redis URL or the allowed-domain list is missing. The search configuration therefore still gates the package and advisory routes. That coupling must be removed regardless of the selected search backend. The separation below is that requirement: specified and not yet implemented.

```
@dataclass(frozen=True)
class ToolDependencies:
    search: SearchBackend | None
    extract: ExtractBackend
    package: PackageRegistryBackend
    advisory: AdvisoryBackend

def build_tool_dependencies(settings: GatewaySettings) -> ToolDependencies:
    return ToolDependencies(
        search=build_search_backend_if_configured(settings),
        extract=BoundedHttpExtract(...),
        package=PublicPackageRegistries(...),
        advisory=OsvAdvisoryClient(...),
    )
```

Route registration is capability-specific:

Capability	Availability rule (target)	At 91fd64a
Search	Selected backend configured and live acceptance gate passes	Tavily when `TAVILY_API_KEY`, tool Redis URL and allowed domains are set
Extract	Bounded HTTPS fetch and prior-search provenance support available	Same bundle; unavailable without the search configuration
Package lookup	Always available with public PyPI/npm/Maven clients	Same bundle; unavailable without the search configuration
Security advisory	Always available with public OSV client	Same bundle; unavailable without the search configuration

Do not delete the Tavily adapter until replacement acceptance tests and rollback review pass. Record that a standalone search backend may need to return if the aggregator search path is withdrawn. This finding is recorded in the correction map for assignment to an owning GitHub issue; this edition commissions no implementation.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

9D. Gateway Server-Side Policy Revalidation

The Gateway independently revalidates every privileged input. Agent-side checks are defense in depth and never authoritative.

Control	Gateway enforcement
Allowed domains	Intersect caller request with local allowlist; forward effective policy; reject returned URLs outside it
Extract provenance	Require exact prior search result for the same `run_id`; revalidate redirects and final URL
Capacity and accounting	Enforce the complete request guard under trusted policy; retain actual usage and explicit unknowns. Product monetary stops are removed only by the released Task 11 migration. Evaluation spend limits are separately scoped.
Call caps	Key by run and tool; do not share a paid-search configuration gate
Tool policy	Recheck tool class, policy signal, execution mode, and authenticated local subject
Usage	Reject missing, malformed, negative, or unparseable provider-reported cost

```
def authorize_tool_call(request, *, policy, ledger):
    effective_domains = policy.intersect_domains(request.domains)
    policy.require_tool_allowed(request.tool, request.execution_mode)
    policy.require_call_capacity(request.run_id, request.tool)
    require_final_capacity_and_output(request, policy)
    return AuthorizedToolCall(
        request=request,
        effective_domains=effective_domains,
    )
```

There is no org/project dimension. Cross-run spend policy remains `P9.85-FU-3` and is not added by this correction.

9E. Evidence Ledger Schema

Evidence entries retain structural provenance and provider-reported usage without adding a second normalized-credit charge.

```python
class EvidenceLedgerEntry(BaseModel):
    evidence_id: str
    run_id: str
    session_id: str
    request_id: str
    gateway_request_id: str
    provider_request_id: str | None
    provider: str
    model: str | None
    model_version: str | None
    cache_hit: bool
    billing_units: int
    cost_usd: Decimal
    source_url: AnyHttpUrl | None
    trust: Literal["trusted", "untrusted"]
    policy_reason: str
    recorded_at: datetime
```

`cost_usd` is copied from validated GatewayUsage. `billing_units` remains provider-native accounting normalized only to a stable integer field. Package/advisory operations may record zero billable units without inventing vendor charges.

Reconciliation methods are `total_cost_usd()` and `total_billing_units()`. Credit-named fields and `total_credits()` are removed under the separate USD rename.

9E.1 Protocol-visible USD rename (implementation complete; real-client evidence UNRUN)

The run-total field exposed in ACP result payloads for search, extract, package and advisory responses was a wire contract, so its rename from the credit-named `ledger_run_total_credits` was a contract migration, not an internal refactor.

Status at 91fd64a: the rename is implemented. `optimus/acp/dispatcher.py` emits `ledger_run_total_cost_usd` and `ledger_run_total_billing_units`; `EvidenceLedger` reconciles with `total_cost_usd()` and `total_billing_units()`; no credit-named ledger field remains under `src/`. The only remaining `credits` token is the provider-native unit label `tavily_credits` used for Tavily usage, which is accounting metadata, not a charge. Verified by code inspection and the unit and integration tests (`tests/unit/acp/test_dispatcher.py`, `tests/integration/evidence/test_mocked_evidence_flow.py`); the predecessor's real-client obligation is UNRUN because the `requires_acpx` tier was deselected in the sealed runs (Test Strategy §15.5, §16).

The migration's required evidence, with status at 91fd64a:

Record	Required evidence	Status
Search/extract response	`GatewayUsage` and run-total USD reconcile	Unit and integration tests
Package/advisory response	Free operation remains non-fabricated and the run total is stable	Unit and integration tests
EvidenceLedger	Decimal-safe sum by run/session/request identity	Unit tests
ACP schema	USD field documented; no compatibility shim for the credit-named field remains at 91fd64a	Code inspection
Real ACP client	Response parses without project-authored client assumptions	UNRUN (`requires_acpx` tier deselected in the sealed runs; Test Strategy §15.5)

The ledger remains append-only and rejects a duplicate charge for the same `gateway_request_id`. The rename changed no USD semantics and added no cross-run policy.

10. Usage Accounting Service and TimeSeries Policy

`UsageAccountingService` accepts only validated GatewayUsage/ProviderUsage records. It persists provider-reported billing units, tokens/cache details, model/version, and `cost_usd`.

```
# Illustrative shape. The implemented method is UsageAccountingService.record_gateway_usage
# in src/optimus/usage/accounting.py; it enforces the same validation before appending.
def record_usage(self, usage: GatewayUsage, *, context: RequestContext) -> None:
    require_nonempty(usage.gateway_request_id)
    require_nonnegative_decimal(usage.cost_usd)
    require_nonnegative_int(usage.billing_units)
    self.ledger.append(
        ProviderUsage.from_gateway_usage(usage, context=context)
    )
```

It never substitutes a locally calculated charge when provider cost exists. Missing, null, negative, or malformed cost fails closed before generated content or evidence is applied.

A versioned price snapshot may remain as labeled diagnostic/comparison metadata only. It cannot overwrite provider-reported cost or silently supply a release-grade value.

Required identity

- `run_id`
- `session_id`
- `request_id`
- `gateway_request_id`
- `provider_request_id` when returned
- aggregator and actual provider when returned
- requested alias and resolved model/version

10.1 RedisTimeSeries persistence

RedisTimeSeries retention and idempotent creation mechanics remain unchanged.

```
TS.CREATE optimus:usage:<run_id>:cost_usd
  RETENTION 2592000000
  LABELS run_id <run_id> metric cost_usd

TS.CREATE optimus:usage:<run_id>:billing_units
  RETENTION 2592000000
  LABELS run_id <run_id> metric billing_units
```

Use `TS.ALTER` when an existing series needs required labels or retention correction. Never drop existing measurements during schema alignment.

Reconciliation

For each accepted Gateway request:

```
Gateway response cost_usd
  == ProviderUsage.cost_usd
  == EvidenceLedger entry cost_usd, when evidence-producing
  == RedisTimeSeries appended value
```

Duplicate `gateway_request_id` records must be idempotent or rejected before a second charge is written. Price snapshots are diagnostic only. Observability export is not a billable usage record.

10A. Provider Usage Ledger and Observability Export

ProviderUsage persists the wire-level GatewayUsage fields verbatim and adds reconciliation attribution: aggregator, actual provider, resolved model/version, service, native unit, run/session/request IDs, and an optional diagnostic price-snapshot ID.

It does not add an Optimus credit charge. `cost_usd` is the upstream-reported aggregator charge. Public package/advisory calls record operational evidence without invented cost.

OpenTelemetry export

The agent sends authenticated structured trace events to `/v1/observability/traces`. The Gateway:

1. validates the event schema and local subject;
2. redacts credentials and sensitive content;
3. maps fields to OTel spans/events;
4. exports OTLP;
5. records delivery status and trace identifiers.

Phoenix is the documented local default. Required attributes include run, session, request, Gateway/provider request, execution mode, generation scope, model/provider, cache, `cost_usd`, billing units, policy/tool, validation, retry, and failure fields.

Trace delivery has no allocated or amortized per-request observability charge.

11. Sprint 1 Implementation Checklist

Trust boundary

- Accept `127.0.0.1`, `localhost`, and `::1`; reject every non-loopback Gateway URL.
- Remove production-mode, extra-origin, hosted-origin, and tenant-profile bypasses.
- Prove WSL2 agent and Gateway run in the same network namespace.
- Prove the agent process resolves only Gateway URL/shared secret.
- Prove the Gateway credential never appears in agent state, logs, errors, telemetry, or children.

Model transport

- Use OpenRouter as default aggregator and the OpenAI-compatible transport.
- Retain Vercel only if Python integration is modest; otherwise assign a named backlog entry. (Status at 91fd64a: no Vercel integration exists.)
- Remove direct provider branches and `anthropic_client.py`. (Done: no `anthropic_client` module at 91fd64a.)
- Parse provider-reported usage/cost; fail closed when incomplete.
- The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

Tool capability

- Separate package/OSV route construction from paid-search configuration. (Open at 91fd64a: one bundle, gated by `TAVILY_API_KEY`, tool Redis URL and allowed domains.)
- Keep Tavily until replacement acceptance and rollback review pass. (Tavily is the implemented provider at 91fd64a.)
- Prove deterministic annotations, domains, URL revalidation, usage, and plugin live gate. (Spike evidence only, 2026-07-27; no Gateway implementation of the plugin path.)
- Prove extract redirect, SSRF, media, size, time, and provenance controls.

11.1 Accounting, telemetry, and release evidence

Accounting

- Persist provider-reported Decimal-safe `cost_usd` and billing units.
- Preserve run/session/request/Gateway/provider request attribution.
- Remove credit-named fields in the separate wire-aware USD migration.
- Prove RedisTimeSeries retention and idempotent creation/alter behavior.

Telemetry

- Export real OTLP spans to a live Phoenix evidence tier.
- Prove no LangSmith dependency, key, or direct backend egress.
- Record validation, retry, tool-policy, and final-disposition attributes.

Release gate

The agent process has only Gateway URL/shared secret. The Gateway process has only the approved aggregator key after migration acceptance. No direct-provider, Tavily, or LangSmith key remains.

Real evidence tiers must use their named dependencies. ACP protocol evidence uses the independent `acpx` client. Fake-based tests remain unit evidence and cannot justify live sign-off.

The deterministic search plugin is release-blocking while it remains the Phase 1 path. A successor server tool is not accepted until live verified-or-fail evidence proves exactly one search, search-use accounting, valid annotations, domain controls, and fail-closed behavior.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

11A. Test Coverage and Observability Cross-Reference

The Test Strategy v1.9 is authoritative for coverage, release, live-dependency, and trace evidence.

Code coverage gates use the Optimus product group and separate context_engine / optimus_model_policy 80% floors; all-source aggregate is informational, with safety-critical modules protected from regression. OTel/OTLP trace assertions validate debugging and regression fields but do not count toward code coverage.

Evidence class	Required dependency
Unit	Fakes allowed; no network/I/O unless intrinsic
Redis integration	Real TimeSeries-capable Redis
Gateway live	Real Optimus credentials and live local Gateway
ACP protocol	Independent `acpx` client
Trace live	Real OTLP export to Phoenix
Release	Agent/Gateway credential and egress scopes proven separately

LangSmith trace assertions and amortized observability accounting are deleted.

Target contract after the named trusted-policy / Task 11 / coverage migration as applicable; the dated inactive production baseline is stated on the cover.

12. Guardrail and Workflow Component Contracts

All model-touching guardrails use the same loopback Gateway, developer aggregator account, actual accounting and explicit evaluation caps, provider-reported cost, and OTel trace path.

This section specifies the Phase 1 enforcement and workflow components introduced as a cross-cutting concern in HLD §13. The authoritative policy rationale lives in the companion Agent Execution Guardrails & Workflow Strategy v1.4 (§2-§9); the representative Pydantic type shapes are in its §10.2. The contracts below are owned by this section. All model-touching elements (the borderline classifier and the loop completion evaluator) route through the Optimus Gateway under the existing developer aggregator account and normalized ledger (§0A, §10A); the guardrails introduce no second cost path.

12A. Permission & Pre-Tool Enforcement

PermissionPolicy evaluates mode -> user_deny -> project_allow -> impact -> classifier and returns a single PermissionDecision (ALLOW | DENY | HOLD); deny always precedes allow; the classifier cannot overturn a deny.

PermissionDecision contains verdict, layer, rule_id, reason, and requires_human_approval.

PreToolGuard fires after the agent assembles a tool call, before execution (cf. PreToolUse), for every tool class (bash, file_edit, mcp_call, web); deterministic checks run before any classifier. It returns PreToolResult.

CommandSafetyValidator is a local, in-process shell validator covering destructive commands, pipe-to-shell, environment/credential access, Unicode homoglyph/confusable characters, ANSI/control sequences, insecure transport, and unexpected network egress. Satisfiable by Tirith or an equivalent validator; the design takes no hard dependency on any one implementation. Implemented in-process in `optimus/guardrails/command_safety.py` at 91fd64a; Tirith is not a dependency.

ToolInvocationAuditEvent is an append-only record of each decision (verdict, layer, failed checks, approver) feeding the same trace sink as evidence and cost telemetry.

12B. Prompt-Injection & MCP Supply-Chain Trust

MCPTrustRegistry captures server_id, manifest_hash, allowed_tools, permission_scope, and approved. MCP servers are never auto-loaded from cloned repositories; a manifest-hash change forces re-approval; tool descriptors (name/description/schema) are inspected for injection before being surfaced to the planner.

ConfigTrustScanner treats agent config and rule files as code, scanning on ingest for embedded instructions, exfiltration endpoints, and homoglyph/ANSI content before they may influence behavior.

12C. Bounded Agent Loops

GoalLoopController, IterationState, CompletionEvaluator, ProgressLedger, LoopBudgetPolicy (max_iterations, max_wall_clock_minutes, repeated_failure_limit; `max_budget_usd` defaults to None and is set only by an explicitly capped evaluation caller; the model default for repeated_failure_limit is 3 in `optimus/loops/models.py` and ACP planning composes 2), and LoopStopReason (COMPLETED | MAX_ITERATIONS | WALL_CLOCK | REPEATED_FAILURE | HUMAN_HALT; BUDGET_EXHAUSTED only for independent evaluation) are the bounded-loop contracts.

Persistent state lives in files, git history, task manifests, traces, and the evidence ledger (§9E), not in an ever-growing chat context. The pre-tool guard (§12A) is never bypassed inside a loop, and the completion evaluator is a reserved cheap Gateway-routed capability. This section does not claim it is active; unknown costs do not latch a product goal loop. Task 11 removes the product monetary predicates on the reviewed cf9bb9d branch. Independent evaluation monetary policy remains explicit.

12D. Curated Workflow Skills

`SkillRegistry`, `SkillManifest` (`name`, `description`, `globs`, `allowed_tools`, `owner`, `version`, `trust_level`), `SkillTrustPolicy` (draft/untrusted skills blocked in Agent mode), and `SkillInvocationPolicy` define curated workflow skill resolution.

A skill's declared `allowed_tools` are enforced by the pre-tool guard (§12A) - a skill can never widen the agent's tool surface - and a skill can never override project or user deny rules.

13. Context Engine, policy and cost integration contracts

13.1 Source-to-view boundary

Canonical `ConversationTurn` records remain host-owned and sanitized. `make_history_snapshot` in `optimus.context.adapter` creates immutable ordinary turns and protected host projections from committed records. Supply the current sanitized prompt separately. `HistoryRevision` binds session, committed generation, last sequence and source digest. No summary changes canonical records or manufactures effects/approval facts.

`ContextEngine.prepare_view(snapshot, *, strategy, parameters, limits, checkpoint, maintenance, cancelled)` returns a bounded `PreparedView`. `StrategyParameters` carries compaction tail, hybrid anchor/tail, summary output cap, max_maintenance_calls, prompt_version and format_version; all fields participate in its digest. `ViewLimits` carries explicit source, history/maintenance token and transient byte bounds, including required `summary_max_bytes` and optional `maintenance_calls_remaining` (None for standalone use, a finite nonnegative integer supplied by the host; booleans rejected). A valid summary byte bound is at least one and strictly below the transient bound. No deployment values are inferred from dataclass defaults.

The injected estimator is subadditive over any strings the engine concatenates: `estimate(a + b) <= estimate(a) + estimate(b)`. Supported production policy currently describes a verified conservative `ceil(r * UTF8_bytes)` profile with non-negative overhead. The complete engine-assembled text is checked before each callback. ViewLimits.maintenance_input_tokens=131072 bounds that text only; the host summary prompt and wire framing are additional, and the complete Gateway request plus reserved output has its own route-capacity check. A downloaded tokenizer or fixture ratio does not silently become a new registry estimator mechanism.

Released offline numeric set (verified route still required)

Where these values live at 91fd64a: the reviewed test profiles and test fixtures (`optimus_model_policy/test_profile_qualification.yaml` output reserves 32768/8192; `tests/` fixtures), not shipped production defaults. `MAX_CONTEXT_CEILING_TOKENS = 262_144` (`optimus_model_policy/registry.py`) and `context_ceiling_tokens: 262144` (`defaults.yaml`) are the ceiling constants; `CONVERSATION_MAX_BYTES = 524_288` (`optimus/acp/conversation.py`) is the absent-engine source floor.

Field	Proposed value / unit
Absent / attached source	524288 / 1048576 UTF-8 bytes
Prompt-time record reservation	131072 bytes; not a reply cap
Transient engine input	524288 bytes
History allowance	min(131072 tokens, floor(0.8 * usable_input)); protected state first
Engine maintenance text	131072 estimated tokens; host prompt/wire framing extra
Summary output / bytes	S <=8192 tokens; summary_max_bytes=floor(S/r)
Compaction tail	32768 tokens
Hybrid anchor / tail	16384 / 65536 tokens, deduplicate overlap
Implementer output	32768 tokens, enforced actual route cap
Effective total	min(262144, verified window), includes output once
Maintenance / repacks	18 shared logical callbacks; at most one repack after initial view

Compaction summarizes outside its exact whole-turn tail; 131072 is a history ceiling, not a delayed trigger. At verified r=1 these budgets behave largely as byte budgets, less estimator/framing overhead; exact counting is deferred. The named D7 exception is accepted (ADR-018, operator source S4): keep the 524288-byte source floor/warning but refuse an over-capacity complete request readably with OPEN disposition, on absent-engine and attached full-history fallback. No provider guard exemption.

13.2 Whole-turn maintenance and checkpoints

Pack complete rendered turns against both conservative per-piece token sums and exact UTF-8 bytes. Count every separator and header. Reserve actual bytes/tokens for a reused prior summary; reserve the effective summary token cap and maximum summary bytes for each future rolling summary. Admit equality, close a chunk before either bound is exceeded, and preflight the entire finite chunk plan before the first callback. If even one required whole turn cannot fit, fail without starting maintenance for that plan.

Fresh and reused summaries pass the same token/byte/format validation. Rebuild an over-bound checkpoint from canonical source; a rejected rebuild publishes nothing. For supported byte-ratio profiles derive `S = min(summary_output_tokens, maintenance_output_tokens)` and `summary_max_bytes = floor(S / r)` with exact arithmetic. Invalid field relationships are configuration conflicts, not reasons to increase admission silently.

Checkpoints contain host-issued covered turn IDs/digests, strategy/parameter/prompt/format identity and source revision. Reuse requires an unchanged covered prefix, checked turn by turn. Cancellation, stale revisions, changed source/configuration, malformed summaries and partial results never publish a checkpoint. Coverage metadata is created by the host, not trusted from model prose.

Typed unavailable/unsupported maintenance status maps to `maintenance unavailable`; actual failure and rejected output retain attempt receipts and their own failure reasons. The filed C1 message map, common OPEN ending, fallback notice and capacity-recovery wording remain exact. A maintenance cancellation can still yield the older internal `maintenance_failed` diagnostic while the host returns cancelled; record this nonblocking residual without commissioning a diagnostic-code change in C1/C2.

13.3 Host attachment and final request

`ContextAttachment` in `optimus.context.assembly` is frozen and requires the engine, parameters, limits, captured source/route identity, receipt sink and finite max_repacks. The released offline setting is one repack after initial packing. Keep StrategyParameters and its checkpoint digest unchanged across the turn. Pass ViewLimits.maintenance_calls_remaining = max(0, 18 - callbacks_already_consumed) for initial preparation and each repack; this execution-state field is outside summary identity. The engine preflights the complete chunk plan against min(parameters.max_maintenance_calls, remaining) before any callback. Zero remaining permits a valid zero-call reused/sliding view and refuses a plan needing calls. Reuse still requires matching source, coverage, estimator and token/byte bounds. Retain the host defensive counter and allowance classification; repack checkpoints remain unpublished.

Normal production bootstrap remains inactive at 91fd64a as at cf9bb9d: no production path constructs `ContextAttachment` (its docstring records this), `ENFORCEMENT_ACTIVE = False` and `trusted_snapshot()` returns `None`. Production activation requires, each with its own evidence and authority: an eligible route with verified estimator and output facts; a genuine summary-quality receipt; effect and cancellation evidence on the production base; trusted launch snapshot binding; bounded maintenance and repacking proof; and real-client evidence. Test-profile acceptance is not production bootstrap approval. At cf9bb9d the reviewed named test composition supplies explicit process-scoped construction through the real trusted entrypoint and passes attachment/route/receipt dependencies into AcpDuplexAdapter. GatewayModelPolicy is likewise mandatory for the test process. Normal bootstrap enforcement is inactive and the default maps to claude-haiku-4.5; production activation and default removal remain held. See test-composition-contract.md for profiles A/B and approval/hash binding.

An `AttachedTurn` captures history/revision, mode/strategy/parameters, route/model/registry and request identity before any interleaving await. Setters affect later turns. It exposes separate `current_prompt`, exact `selection_text` and inert `conversation_envelope`. Do not hold a configuration lock across provider calls or approval.

`probe_floor` measures full committed canonical history plus provisional prompt without changing disposition. If the engine is unavailable and that projection is <=524288 bytes, prepare full-history fallback with the filed notice and still apply the final complete-request capacity guard. Above the floor, refuse before answer/planning dispatch and keep the thread OPEN. Attached storage class persists through health faults; repeated faults never latch CAP_CLOSED.

Bind the admitted request/view digest to the stored plan. Approved application reuses it and the exact artifact hash; it neither summarizes again nor reselects a model. A changed target requires existing freshness/replan/approval handling. Projection of an old approval never approves a revised plan.

13.4 Registry and final Gateway enforcement

Strict YAML rejects duplicate/non-string keys, aliases, anchors, merge keys and custom tags. Packaged defaults plus operator overrides yield one immutable, canonically hashed `RegistrySnapshot`. Formatting changes alone do not alter its effective identity. Neutral policy code imports no host runtime. Trusted launch composition supplies the actual validated object to host and Gateway; a caller-supplied hash is not authority.

Per-role eligibility rejects verified window <262144 or maximum output below the role reserve (ROUTE_WINDOW_BELOW_CEILING / output-cap eligibility). It requires explicit assignment, capabilities, exact endpoint/quantization, verified window/output/parameter/estimator facts, and any required summary receipt/disclosure. Unknown route facts and unknown trusted snapshot mean zero upstream calls. Fixing model aliases does not permit direct provider keys. The legacy host permits an initial attempt plus at most three retries. The inactive-policy Gateway separately permits at most three upstream attempts, including its legacy transient timeout/URL/5xx retries. Once enforcement is active, the Gateway permits one primary attempt and at most one identical-payload resend on the same route, only after a definitely NOT_SENT result (including a proven pre-connection timeout) or rejected HTTP 429. A timeout after connection, uncertain delivery, 5xx or unknown accounting is not resent. The host honors non-retryable classifications so it cannot multiply attempts. No alternative-model recovery is added. Each physical attempt retains its own receipt.

At the final provider boundary enforce `complete_input_estimate + output_reserve <= min(262144, verified_route_window)`, `0 < output_reserve <= verified_route_max_output`, and the exact upstream cap/finish semantics. Count all messages, framing, tools/files/evidence and reasoning reservations; count output once. A complete-request tokenizer may inform route verification, but production use requires a registered supported and reviewed mechanism; neither sample ratios nor broad tokenizer-family labels suffice.

A WRITE-bearing result is usable only with a verified complete successful finish. Length-limit, missing or unknown finish fails before parse/store/approval/execution, even if the prefix parses. Chat incomplete output is visibly incomplete. Preserve receipts for rejected/discarded/cancelled work.

13.5 Exactly-once settlement and D6

`StageReceipt` identifies session, turn, stage, attempt, available Gateway/provider request IDs, requested/resolved model/provider, captured configuration/registry, cost or explicit unknown, token/cache data and time. Turn/scope summaries retain known subtotal, unknown IDs, receipt identities and completeness. Duplicate identical receipts are idempotent; conflicting duplicates are integrity errors. Actual attempts count once even when their summary/view/result is stale or discarded. Applying a stored plan never charges its old planning again.

Strict Gateway usage validation remains: invalid cost is not a valid zero-cost `ProviderUsage`. Record the attempted stage as unknown and refuse its unverified successful result as required; keep known subtotals before/after it. Unknown accounting does not introduce a session-wide product monetary stop or automatic retry. Telemetry must not debit an `agent_run` projection a second time.

Configured per-turn/session alerts emit once per actual threshold crossing, with stable scope/crossing identity. This draft invents no operator alert amount. Daily totals require durable reconciliation and explicit IANA timezone/day identity; absent prerequisites make that scope unavailable. P9.85-FU-3 and P11.26-CAND-2 retain existing ownership. Operational public requests and trace export do not acquire fabricated provider charges.

13.6 Product/evaluation policy migration

The reviewed Task 11 implementation removes product dollar predicates from Agent planner/runner and non-ACP goal-loop paths, positive-dollar construction requirements and stop-bearing telemetry/planner prompts. Use an explicit optional evaluation monetary policy only for independently capped callers. Keep product accounting and count/time/failure/cancellation/security/capacity controls. No renamed limit, hidden maintenance subtraction or large/infinite sentinel is acceptable.

The three strict xfails at dea4624 were held cost-stop-removal evidence. Task 11 now satisfies those assertions at cf9bb9d and removes their markers; the final full runs verify the product/evaluation split, and the change was merged to `main` via PR #276 (`c509fc8`). A result costing more than the former $0.05 must still be delivered successfully on a product path; an

independently capped evaluation must stop under its separate envelope. Preserve the current three planning rounds, 30-minute limit, repeated-failure limit two and single-call Chat.

13.7 ACP projection and meter

Publish one full-set configOptions projection that preserves model/mode/strategy options and pinned option/setter IDs. Show the strategy picker only when configured attached, retaining it through engine faults. Capture each turn's configuration before awaits; use one revision/resync mechanism so an old publication cannot clear newer pending state. Sliding's filed omission disclosure appears on the first admitted sliding turn and reports actual delivery.

The meter records at local Gateway dispatch before its response, using the largest dispatched planning/answer input with that request's usable capacity, choosing smaller capacity on ties. Maintenance cost contributes to cost records, not the ring. A host capacity refusal yields no fabricated reading. A non-capacity Gateway preflight rejection can leave a reading for a request sent locally but never sent upstream; this meter is not proof of provider delivery. Distinguish storage warnings, strategy-unavailable fallback and whole-request capacity refusal.

13.8 Effect proof and rollout

Maintain Task 2's real WRITE/TEST registration, start gates, terminal evidence and cancellation behavior. A normally returned TEST is executed even when its test exit is nonzero; test verdict and operation terminal are distinct. Denied/not-started work is reflected in a fresh consistent effect snapshot before commit. Already-started operations are not promised to be interrupted. Never reconstruct authority from mutation counts or model prose.

Verify fresh-session rollout on the actual production base. If durable replay/resume arrives before rollout, its owner must supply reviewed provenance before old placeholder effects are treated as authoritative. Keep the sandbox's unsynchronized repair gap disclosed until its separately authorized synchronization. The migration adds no durable-history mechanism.

13.9 Implemented capacity outcomes and test launch

Once trusted enforcement is active, only INPUT_EXCEEDS_CAPACITY selects the absent-engine capacity text; CAPACITY_REFUSED without a known reason stays generic. Preserve known costs from earlier requests in the turn. The rejected request itself has zero upstream attempts.

Chat could not answer because this request exceeds the model's input capacity. This request was not sent to a model. A shorter prompt or a narrower request involving fewer workspace files may help. If earlier conversation history is the cause, start a new thread. This thread stays open.

Planning stopped because its next request exceeds the model's input capacity. That request was not sent to a model. A shorter prompt or a narrower request involving fewer workspace files may help. If earlier conversation history is the cause, start a new thread. This thread stays open.

The request meter uses complete packed input/usable capacity from the final policy estimator. Warn at 80% once per session; a jump over capacity emits refusal directly. Storage’s 80% notice remains separately labelled storage.

The source-owned qualification/attached profiles are selected by name only. HMAC launch authorization binds the security snapshot containing the effective profile hash; host and Gateway independently compose it, and the authenticated child manifest binds the same identity. The host literal check is internal consistency, not a separate operator approval. A profiled host must spawn its own profiled Gateway. Already-listening/external Gateway, no child, --no-auto-start, --framed and --strict are refused; a mismatched explicit model refuses before Redis/Gateway/server. No run-gateway profile flag is needed.

14. Contract status: implemented, verified, active (correction of 2026-10-15)

This section classifies the executable contracts this document specifies at the inspected source `91fd64ac78025efe1076bddb106ff89e6adb2030` (origin/main, 2026-10-15). Implementation, verification and activation are independent fields. Conceptual or retained-prototype interfaces are labelled as such where they appear; this table is the collected view. It changes no accepted contract. The correction map filed with the sources (`docs/sources/capability-evidence-correction/correction-map.md`) lists each change and its source reference.

Contract (section)	Implementation at the pin	Verification	Activation
Gateway routes `/v1/responses`, `/v1/chat/completions`, four typed tools, `/v1/observability/traces` (§0, §D)	Implemented (`optimus_gateway/server.py`, `tool_handlers.py`, `observability.py`)	Unit tests; `requires_gateway` and `requires_phoenix` live tiers	Active
Strict loopback settings (`OptimusGatewaySettings`, §9C)	Implemented (`optimus/config/gateway.py`, `_validate_loopback_gateway_url`; the `validate_loopback_gateway` listing on the carried §9C page is illustrative)	Unit tests	Active
Upstream transport `UrllibOpenAICompatibleClient` (§6, §6.1)	Implemented (`optimus_gateway/upstream_client.py`)	Unit tests	Active
Vercel optional endpoint (§0, §6)	Specified; not established by inspection	None	Inactive
Deterministic OpenRouter plugin search (§9C.1)	Specified target; Gateway search is `TavilyWebToolProvider`	Spike evidence only (2026-07-27, outside the Gateway)	Inactive
Package/OSV construction independent of search configuration (§9C.4)	Specified; not implemented (single bundle gated by `TAVILY_API_KEY`)	None	Not applicable
Gateway server-side policy revalidation (§9D)	Implemented (`optimus_gateway/tool_policy.py`, `tool_handlers.py`); the `authorize_tool_call` listing is illustrative	Unit tests	Active
Evidence ledger and USD run totals (§9E, §9E.1)	Implemented (`optimus/evidence`); rename complete, fields `ledger_run_total_cost_usd` and `ledger_run_total_billing_units`	Unit tests; `requires_redis` tier	Active
Usage accounting (§10)	Implemented (`UsageAccountingService.record_gateway_usage` in `optimus/usage/accounting.py`); the `record_usage` listing is illustrative	Unit tests	Active
RedisTimeSeries retention and idempotent creation (§10.1)	Implemented (`optimus/telemetry/redis_adapter.py`, 2,592,000,000 ms, `TS.CREATE` then `TS.ALTER`)	Unit tests; `requires_redis` tier	Active

Contract (section)	Implementation at the pin	Verification	Activation
Retry policy: host initial attempt plus three retries; enabled Gateway at most two physical attempts (§4A, §6)	Implemented (`optimus/retry/policy.py` `max_retries = 3`; `MAX_ROUTE_ATTEMPTS = 2` in `optimus_model_policy/binding.py`)	Unit tests	Host policy active; the two-attempt rule applies once enforcement is active
ExecutionMode PLAN/CHAT/AGENT and mutation assertion (§4)	Implemented (`optimus/runtime/modes.py`; `MutationGuard` and `assert_mutation_allowed` in `optimus/runtime/mutation.py`, used by `optimus/gates/mutation_flow.py`)	Unit and integration tests	Active
`CodeGenerationScope`, `ExecutionStrategy`, `RigorLevel`, `AssumptionLedger`, `ToolResponse`, `ASTOptimizationInput`, `ADLParserStrategy` (§4, §5)	Retained design contracts; not present as named types under `src/` (`MULTI_FILE_CHANGESET` exists as a permission impact class)	None as named	Inactive
Composite fitness engine scaffolding (§7, carried page)	Illustrative scaffolding; the implemented runner is `optimus/gates/fitness.py`; the placeholder architecture and metric gates have no PyTestArch or maintainability-index implementation	Unit tests of the runner	Active for composition-supplied checks
Guardrail contracts: `PermissionPolicy`, `PreToolGuard`, `CommandSafetyValidator`, `ConfigTrustScanner`, `MCPTrustRegistry` (§12A, §12B)	Implemented (`optimus/guardrails`)	Unit tests (`tests/unit/guardrails`)	Active
Borderline classifier and loop completion evaluator model calls (§12A, §12C)	`GatewayCompletionEvaluator` implemented (`optimus/loops/completion.py`) behind `CompletionEvaluatorProtocol`; production composition uses `DeterministicCompletionEvaluator` and nothing in `src/` constructs the Gateway evaluator; classifier reserved	Unit tests (`tests/unit/loops/test_completion.py`); none live	Inactive (not composed in production)
Bounded loops: `GoalLoopController`, `LoopBudgetPolicy`, `LoopStopReason` (§12C)	Implemented (`optimus/loops`, `optimus/agent/planning_loop.py`); `max_budget_usd` defaults `None`	Unit and integration tests	ACP planning bounds active (3 turns, 30 minutes, 2 repeated failures)
Curated skills: `SkillRegistry`, `SkillManifest`, `SkillTrustPolicy` (§12D)	Implemented (`optimus/skills`)	Unit tests	Active
`make_history_snapshot`, `HistoryRevision`, `ContextEngine.prepare_view`, `StrategyParameters`, `ViewLimits` (§13.1)	Implemented (`optimus/context/adapter.py`, `src/context_engine`)	Offline-verified at cf9bb9d; unit tests at the pin	Engine usable; production attachment inactive

Contract (section)	Implementation at the pin	Verification	Activation
Whole-turn maintenance, checkpoints, `maintenance_calls_remaining`, `summary_max_bytes` (§13.2)	Implemented (`context_engine/engine.py`, `checkpoints.py`, `contracts.py`)	Offline-verified at cf9bb9d	Inactive in production
`ContextAttachment`, `AttachedTurn`, `probe_floor`, `max_repacks` (§13.3)	Implemented (`optimus/context/assembly.py`); no production constructor of `ContextAttachment`	Offline-verified at cf9bb9d; named test composition only	Inactive
Trusted `RegistrySnapshot`, `ROUTE_WINDOW_BELOW_CEILING`, final capacity guard (§13.4)	Implemented (`optimus_model_policy`)	Offline-verified at cf9bb9d	Inactive: `ENFORCEMENT_ACTIVE = False`, `trusted_snapshot()` is `None`
`StageReceipt` exactly-once settlement and D6 (§13.5, §13.6)	Implemented; product dollar stops removed, merged via PR #276	Offline-verified at cf9bb9d	Active
ACP projection, meter and capacity messages (§13.7, §13.9)	Implemented (`optimus/acp/spec.py`, `gateway/attempts.py`)	Offline-verified at cf9bb9d	Capacity text selectable once enforcement is active
`--plan12-test-profile`, `TEST_PROFILE_GATEWAY_NOT_STARTED`, `GatewayModelPolicy` (§13.9)	Implemented (`optimus/acp/__main__.py`, `optimus_model_policy/test_profiles.py`, `optimus_gateway/model_policy.py`); profile A admits no model, profile B is declared unavailable	Offline-verified at cf9bb9d with a synthetic eligible profile	Test-composition only
Package extraction and publication of `context_engine` as a wheel (§13.1)	Buildable; independent wheel proof at cc6959b carried to cf9bb9d over unchanged inputs	Historical proof only; not rebuilt at the pin	Not published

Activation prerequisites, restated. Production attachment of the Context Engine requires, each with its own evidence and authority: an eligible route with verified estimator, output-cap and finish facts; a genuine summary-quality receipt; effect and cancellation evidence on the production base; trusted launch snapshot binding; bounded maintenance and repacking proof; and real-client evidence. Test-profile acceptance is not production bootstrap approval. Unimplemented integration points remain future work under their existing owners (Plan 12.2, issue #237); this edition commissions none of them.